

APLICAȚIA 2

IMPLEMENTAREA UNUI CIRCUIT ÎN DISPOZITIVE FPGA

1. Rezumat

Acest laborator își propune introducerea în tehnologia Intel/Altera și ModelSim/QuartaSim. Vor fi prezentați o serie de pași necesari în toate lucrările de laborator care vor urma.

Vor fi trecuți în revistă etapele realizării unui design digital (descriere, simulare și prototipaj) folosind ca și produse software ModelSim/QuartaSim și Quartus. Circuitul descris este un circuit *majority voting* cu 3 intrări. Se va folosi ModelSim pentru simulare și Quartus pentru sinteză, post-sinteză și analiză de timp. Lucrările de laborator viitoare tratează circuite mai complexe. Scopul principal al acestui laborator este acela de a evidenția etapele de design și maniera concretă de realizare a acestora.

Obiectivele lucrării

Obiectivul acestui laborator este acela de familiarizare cu produsele software ModelSim și Quartus în vederea parcurgerii pașilor de sinteză și implementare a unui design prin realizarea unui design simplu: *majority voter* cu 3 intrări.

Obiective tehnice

1. Familiarizare cu placa Terasic **DE10-Lite**
2. Realizarea unui design simplu: poartă majority folosind Verilog HDL, simularea lui folosind ModelSim
3. Sinteza și implementare design-ului folosind Quartus
4. Configurare FPGA **MAX 10** de pe placa DE10-Lite.

Timp necesar

2-3 ore

Pregătirea pentru laborator

- Citiți documentul înainte de a începe realizarea practică.
- Salvați output-urile pentru fiecare cerință sau anunțați cadrul didactic în vederea prezentării rezultatelor.

Echipamente și Materiale : Acces la software-ul ModelSim și Quartus

Necesar	Cantitate
Software Quartus Prime Lite Edition, cu support pentru dispozitivul (device) MAX10 (https://www.intel.com)	1
Software ModelSim-Intel® FPGA Standard Edition (https://www.intel.com)	1
Placă Terasic DE10-Lite	1

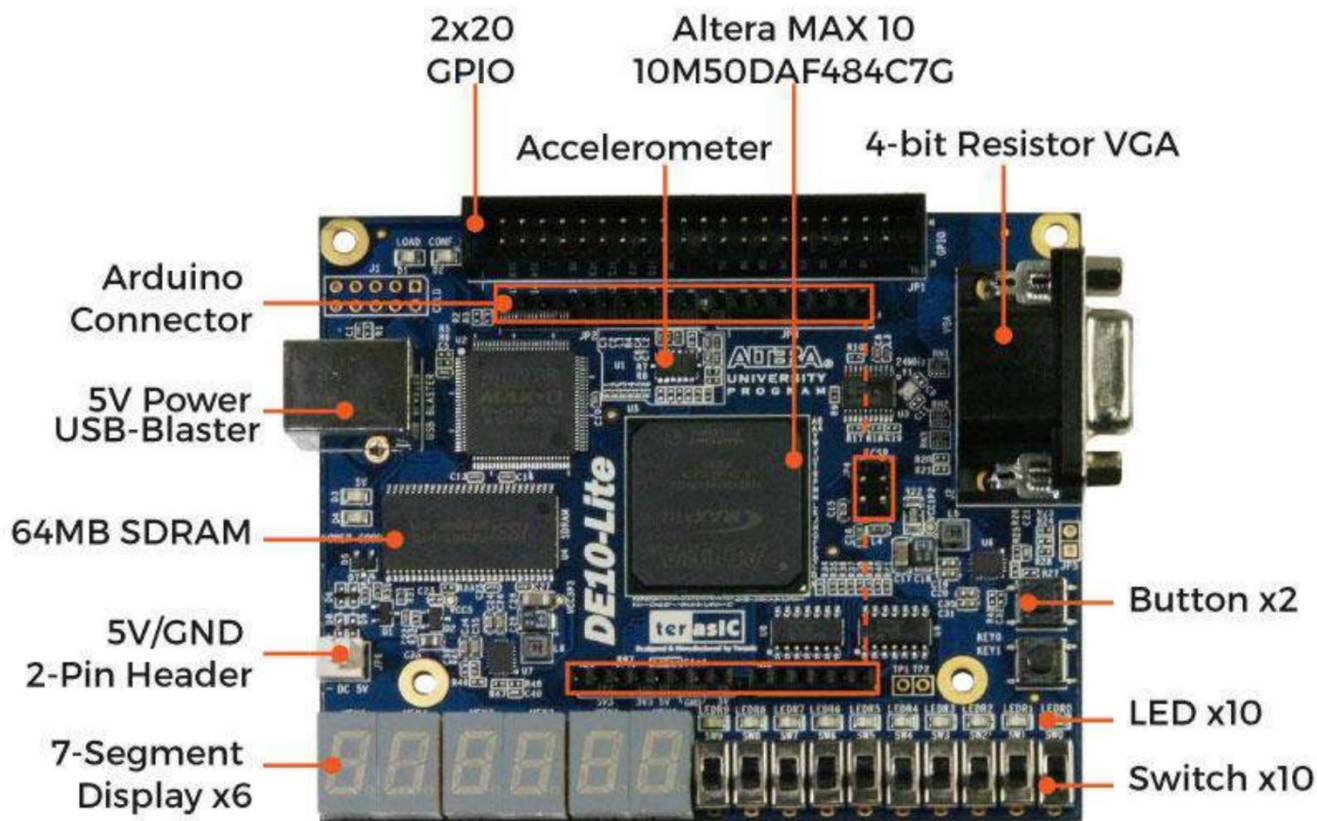
2. Placa DE10-Lite

Documentația aferentă plăcii DE10-Lite o găsiți pe site-ul Terasic:

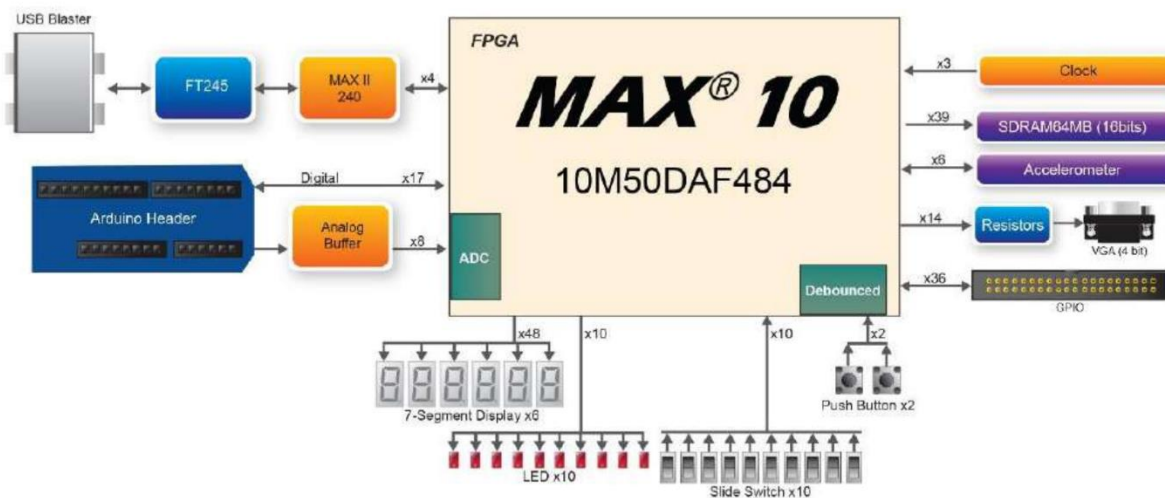
<https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=234&No=1021&PartNo=4>

Dintre caracteristicile plăcii menționăm:

- FPGA-ul:
 - Dispozitiv Intel MAX 10 10M50DAF484C7G FPGA, cu 50K elemente de logica programabila
 - Convertoare Analog-Digital, duale, integrat, fiecare convertor avand 1 intrare analogica si 8 pini cu functii duale
 - 1638 Kbit memorie de tip M9K
 - 144 18bit × 18bit multiplicatoare
 - 4 PLL-uri
- 10 LED-uri
- 10 intrerupătoare (slide switch)
- 2 butoane apasare (push button)
- 6 afisaje cu 7 segmente
- Doua oscillatoare de tact cu frecvența de 50MHz
- USB blaster si conector USB Tip B pentru programare si alimentare
- 64MB SDRAM, x16 bits magistrala de date (data bus)
- VGA cu rezistoare 4-bit
- Accelerometru
- Conectori expansiune



Schema plăcii este prezentată în figura următoare (imaginile plăcii sunt preluate de pe site-ul Terasic).



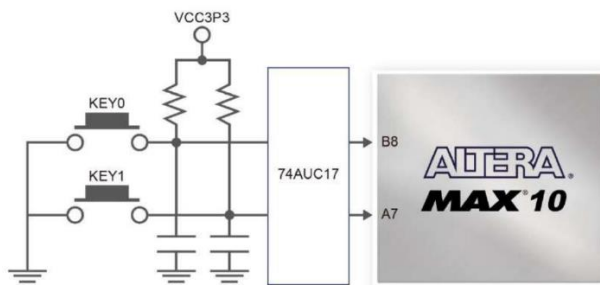
Pentru mai multe informații se recomandă consultarea documentului User Manual:

https://www.terasic.com.tw/cgi-bin/page/archive_download.pl?Language=English&No=1021&FID=a13a2782811152b477e60203d34b1baa

În continuare vom trece în revistă facilitățile de I/O oferite de placa *DE10-Lite*:

- Intrări:

- Butoane (push-button): aceste intrări sunt în mod normal la 1 logic și se setează la 0 logic numai în momentul în care sunt apăsate.



Signal Name	FPGA Pin No.	Description	I/O Standard
KEY0	PIN_B8	Push-button[0]	3.3 V SCHMITT TRIGGER"
KEY1	PIN_A7	Push-button[1]	3.3 V SCHMITT TRIGGER"

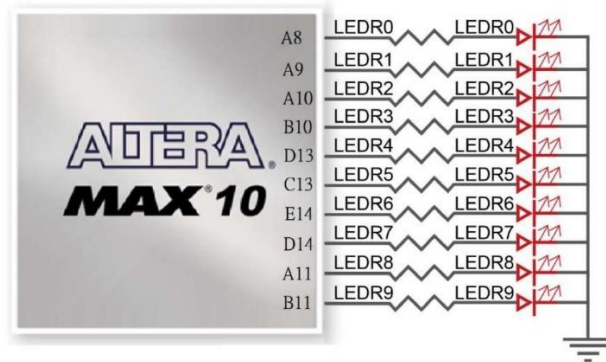
- Întrerupătoare (switch-uri): acestea generează semnale de intrare constante 0 sau 1 logic funcție de poziția întrerupătorului.



Signal Name	FPGA Pin No.	Description	I/O Standard
SW0	PIN_C10	Slide Switch[0]	3.3-V LVTTL
SW1	PIN_C11	Slide Switch[1]	3.3-V LVTTL
SW2	PIN_D12	Slide Switch[2]	3.3-V LVTTL
SW3	PIN_C12	Slide Switch[3]	3.3-V LVTTL
SW4	PIN_A12	Slide Switch[4]	3.3-V LVTTL
SW5	PIN_B12	Slide Switch[5]	3.3-V LVTTL
SW6	PIN_A13	Slide Switch[6]	3.3-V LVTTL
SW7	PIN_A14	Slide Switch[7]	3.3-V LVTTL
SW8	PIN_B14	Slide Switch[8]	3.3-V LVTTL
SW9	PIN_F15	Slide Switch[9]	3.3-V LVTTL

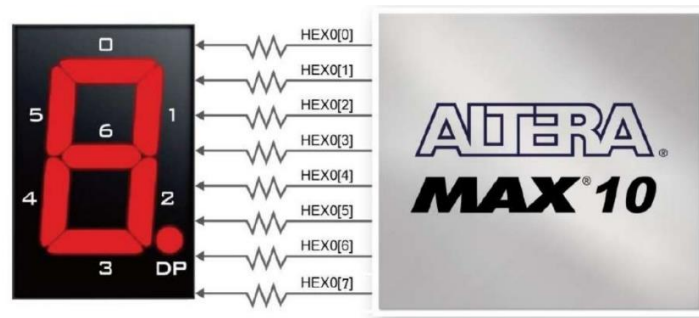
- ieșiri:

- LED-urile: ledul se aprinde la aplicarea 1 logic si stinge la 0 logic.



Signal Name	FPGA Pin No.	Description	I/O Standard
LEDR0	PIN_A8	LED [0]	3.3-V LVTTTL
LEDR1	PIN_A9	LED [1]	3.3-V LVTTTL
LEDR2	PIN_A10	LED [2]	3.3-V LVTTTL
LEDR3	PIN_B10	LED [3]	3.3-V LVTTTL
LEDR4	PIN_D13	LED [4]	3.3-V LVTTTL
LEDR5	PIN_C13	LED [5]	3.3-V LVTTTL
LEDR6	PIN_E14	LED [6]	3.3-V LVTTTL
LEDR7	PIN_D14	LED [7]	3.3-V LVTTTL
LEDR8	PIN_A11	LED [8]	3.3-V LVTTTL
LEDR9	PIN_B11	LED [9]	3.3-V LVTTTL

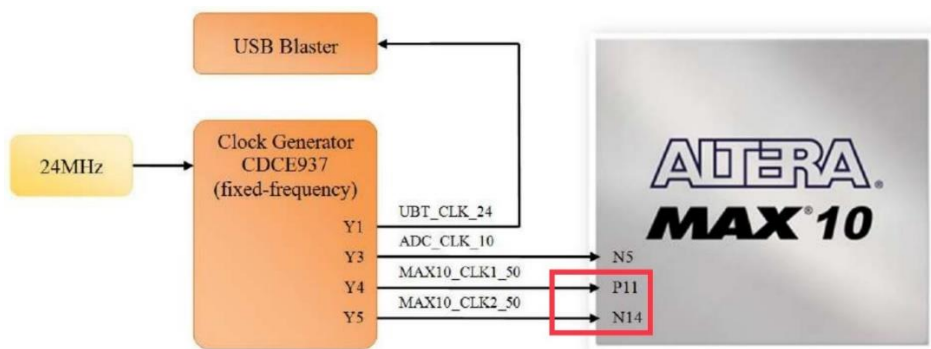
- 6 afișaje cu 7 segmente: aprinderea led-ului aferent se face la 0 logic



Signal Name	FPGA Pin No.	Description	I/O Standard
HEX00	PIN_C14	Seven Segment Digit 0[0]	3.3-V LVTTTL
HEX01	PIN_E15	Seven Segment Digit 0[1]	3.3-V LVTTTL
HEX02	PIN_C15	Seven Segment Digit 0[2]	3.3-V LVTTTL
HEX03	PIN_C16	Seven Segment Digit 0[3]	3.3-V LVTTTL
HEX04	PIN_E16	Seven Segment Digit 0[4]	3.3-V LVTTTL
HEX05	PIN_D17	Seven Segment Digit 0[5]	3.3-V LVTTTL
HEX06	PIN_C17	Seven Segment Digit 0[6]	3.3-V LVTTTL
HEX07	PIN_D15	Seven Segment Digit 0[7], DP	3.3-V LVTTTL
HEX10	PIN_C18	Seven Segment Digit 1[0]	3.3-V LVTTTL
HEX11	PIN_D18	Seven Segment Digit 1[1]	3.3-V LVTTTL
HEX12	PIN_E18	Seven Segment Digit 1[2]	3.3-V LVTTTL
HEX13	PIN_B16	Seven Segment Digit 1[3]	3.3-V LVTTTL

HEX14	PIN_A17	Seven Segment Digit 1[4]	3.3-V LVTTL
HEX15	PIN_A18	Seven Segment Digit 1[5]	3.3-V LVTTL
HEX16	PIN_B17	Seven Segment Digit 1[6]	3.3-V LVTTL
HEX17	PIN_A16	Seven Segment Digit 1[7] , DP	3.3-V LVTTL
HEX20	PIN_B20	Seven Segment Digit 2[0]	3.3-V LVTTL
HEX21	PIN_A20	Seven Segment Digit 2[1]	3.3-V LVTTL
HEX22	PIN_B19	Seven Segment Digit 2[2]	3.3-V LVTTL
HEX23	PIN_A21	Seven Segment Digit 2[3]	3.3-V LVTTL
HEX24	PIN_B21	Seven Segment Digit 2[4]	3.3-V LVTTL
HEX25	PIN_C22	Seven Segment Digit 2[5]	3.3-V LVTTL
HEX26	PIN_B22	Seven Segment Digit 2[6]	3.3-V LVTTL
HEX27	PIN_A19	Seven Segment Digit 2[7] , DP	3.3-V LVTTL
HEX30	PIN_F21	Seven Segment Digit 3[0]	3.3-V LVTTL
HEX31	PIN_E22	Seven Segment Digit 3[1]	3.3-V LVTTL
HEX32	PIN_E21	Seven Segment Digit 3[2]	3.3-V LVTTL
HEX33	PIN_C19	Seven Segment Digit 3[3]	3.3-V LVTTL
HEX34	PIN_C20	Seven Segment Digit 3[4]	3.3-V LVTTL
HEX35	PIN_D19	Seven Segment Digit 3[5]	3.3-V LVTTL
HEX36	PIN_E17	Seven Segment Digit 3[6]	3.3-V LVTTL
HEX37	PIN_D22	Seven Segment Digit 3[7] , DP	3.3-V LVTTL
HEX40	PIN_F18	Seven Segment Digit 4[0]	3.3-V LVTTL
HEX41	PIN_E20	Seven Segment Digit 4[1]	3.3-V LVTTL
HEX42	PIN_E19	Seven Segment Digit 4[2]	3.3-V LVTTL
HEX43	PIN_J18	Seven Segment Digit 4[3]	3.3-V LVTTL
HEX44	PIN_H19	Seven Segment Digit 4[4]	3.3-V LVTTL
HEX45	PIN_F19	Seven Segment Digit 4[5]	3.3-V LVTTL
HEX46	PIN_F20	Seven Segment Digit 4[6]	3.3-V LVTTL
HEX47	PIN_F17	Seven Segment Digit 4[7] , DP	3.3-V LVTTL
HEX50	PIN_J20	Seven Segment Digit 5[0]	3.3-V LVTTL
HEX51	PIN_K20	Seven Segment Digit 5[1]	3.3-V LVTTL
HEX52	PIN_L18	Seven Segment Digit 5[2]	3.3-V LVTTL
HEX53	PIN_N18	Seven Segment Digit 5[3]	3.3-V LVTTL
HEX54	PIN_M20	Seven Segment Digit 5[4]	3.3-V LVTTL
HEX55	PIN_N19	Seven Segment Digit 5[5]	3.3-V LVTTL
HEX56	PIN_N20	Seven Segment Digit 5[6]	3.3-V LVTTL
HEX57	PIN_L19	Seven Segment Digit 5[7] , DP	3.3-V LVTTL

- **Semnalul de tact:** Placa DE10-Lite dispune de 2 oscilatoare de tact (clocks), cu frecventa de 50MHz, conectate la FPGA si care pot fi folosite de catre utilizator.



Signal Name	FPGA Pin No.	Description	I/O Standard
ADC_CLK_10	PIN_N5	10 MHz clock input for ADC (Bank 3B)	3.3-V LVTTL
MAX10_CLK1_50	PIN_P11	50 MHz clock input(Bank 3B)	3.3-V LVTTL
MAX10_CLK2_50	PIN_N14	50 MHz clock input(Bank 3B)	3.3-V LVTTL

3. Etapele realizarii unui proiect/design

3.1 Descrierea

Două modalități:

- Limbaje de descriere hardware HDL (Verilog sau VHDL)
- Folosind scheme (schematic).

Această lucrare ilustrează implementarea circuitului folosind descrierea Verilog HDL.

3.2 Verificarea

În acest pas ne asigurăm că design-ul proiectat funcționează corect:

- Realizarea unui modul de test (sau testbench, prescurtat TB) care stimulează intrările și monitorizează ieșirile
- Simularea și crearea de forme de undă. Vom folosi ModelSim/QuestaSim ca și simulator.

3.3 Sinteza și Implementarea

În acest pas vom folosi software-ul Quartus de la Intel, pentru: traducere, mapare pe componentele FPGA-ului, alocarea componentelor specific dispozitivului FPGA, și rutare în vederea stabilirii interconexiunilor dintre componente. Ieșirea acestui proces este un fișier (.SOF) folosit pentru programarea FPGA-ului.

3.4 Configurarea/ programarea FPGA-ului

Constă în încărcarea fișierului de programare (.SOF) în FPGA.

*Acest laborator prezintă modul în care poate fi realizat design-ul unui circuit digital simplu: **majority voter** cu 3 intrări, folosind Quartus și ModelSim/QuestaSim. În acest laborator sunt realizați toți pașii din procesul de proiectare prezentat anterior.*

4. Descrierea si verificarea unui circuit *majority voting* cu 3 intrări

Pentru realizarea acestui circuit vom folosi limbajul de descriere a hardware-ului Verilog si vom simula design-ul folosind simulatorul ModelSim/QuartaSim.

Vor fi prezentate doua moduri distincte de folosire a simulatorului ModelSim, iar dupa preferinta va fi aleasa una dintre cele 2 metode:

- A: Folosirea simulatorului ModelSim in modul linie de comanda
 - o Editarea fisierelor sursa Verilog se face folosind editorul de text preferat
 - o Rularea simularii se face folosind linia de comanda disponibila in ModelSim
- B: Folosirea simulatorului ModelSim in modul GUI (interfata grafica)
 - o Editarea fisierelor sursa Verilog se va face in editorul simulatorului ModelSim
 - o Comenzile necesare simularii vor fi executate din meniul grafic al simulatorului

4.1 Metoda A: Folosirea ModelSim in modul linie de comanda

Vom crea un director nou pe hard-disk-ul PC-ului unde vom stoca proiectele. Sa presupunem ca el se numeste *wa123* (*wa*=work area). In *wa123*, vom crea un nou director care se numeste, e.g., *ap2* (pentru aplicatia 2).

Nota: se recomanda ca fiecare masa de lucru/grupa sa-si creeze propriul spatiu de lucru, cu nume distinct (e.g., "wa12c" pt grupa 1.2 seria C)

Crearea directorelor se poate face in Linux GUI sau in terminal, folosind comenzi ca *cd* (change directory) si *mkdir* (make directory), e.g.:

```
cd <calea_catre_spatiu_lucru>
```

```
mkdir wa123; cd wa123
```

```
mkdir a2; cd a2
```

Odata ce structura de directoare a fost creata, vom construi sursele Verilog pe care le vom salva in acest nou director (e.g. "*/<cale>/wa123/a2*").

4.1.1 Descrierea design-ului "majority voter"

Folosind editorul de text preferat, vom crea un fisier ce se numeste *majority_voter.v*. Aceasta reprezinta sursa de tip Verilog pentru design-ul nostru: un circuit de votare cu 3 intrari si o iesire. Vom salva sursa in spatiul nou creat, e.g. "*/<cale>/wa123/a2*". Codul aferent este urmatorul.

Evidentiat cu galben sunt liniile de cod care trebuie adaugate pentru a modela comportamentul unitatii "majority_voter".


```

module majority_voter
(
    input in1,
    input in2,
    input in3,
    output maj

);

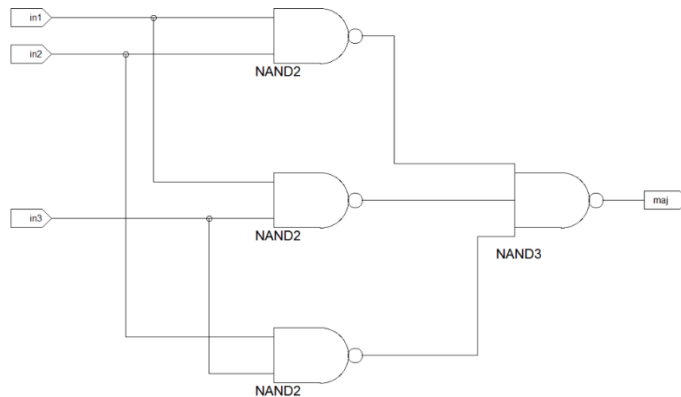
wire nand1, nand2, nand3; //internal wires

assign nand1 = ~(in1 & in2);
assign nand2 = ~(in1 & in3);
assign nand3 = ~(in2 & in3);
assign maj    = ~(nand1 & nand2 & nand3);

endmodule

```

Schema cu porti logice echivalenta este urmatoarea:



4.1.2 Descrierea testului pentru “majority voter”

Pasul urmator este sa creem un fisier de test (sau testbench / tb), tot de tip Verilog, care va folosi la verificarea design-ului. Il vom salva sub numele “majority_voter_tb.v”, in acelasi loc ca si design-ul.

Editam fisierul testbench, dupa cum urmeaza (sunt scose în evidență liniile de cod adăugate pentru generarea stimulilor de intrare):

```

module majority_voter_tb;

// Inputs
reg in1;
reg in2;
reg in3;

// Outputs
wire maj;

// Instantiate the Unit Under Test (UUT)
majority_voter uut (
    .in1(in1),
    .in2(in2),
    .in3(in3),
    .maj(maj)
);

initial begin
    // Initialize Inputs
    in1 = 0;
    in2 = 0;
    in3 = 0;
    // Wait 100 ns for global reset to finish
    #100;
end

always //toggle inputs
begin
    #25 in1 = ~in1;
    #50 in2 = ~in2;
    #75 in3 = ~in3;
end

endmodule

```

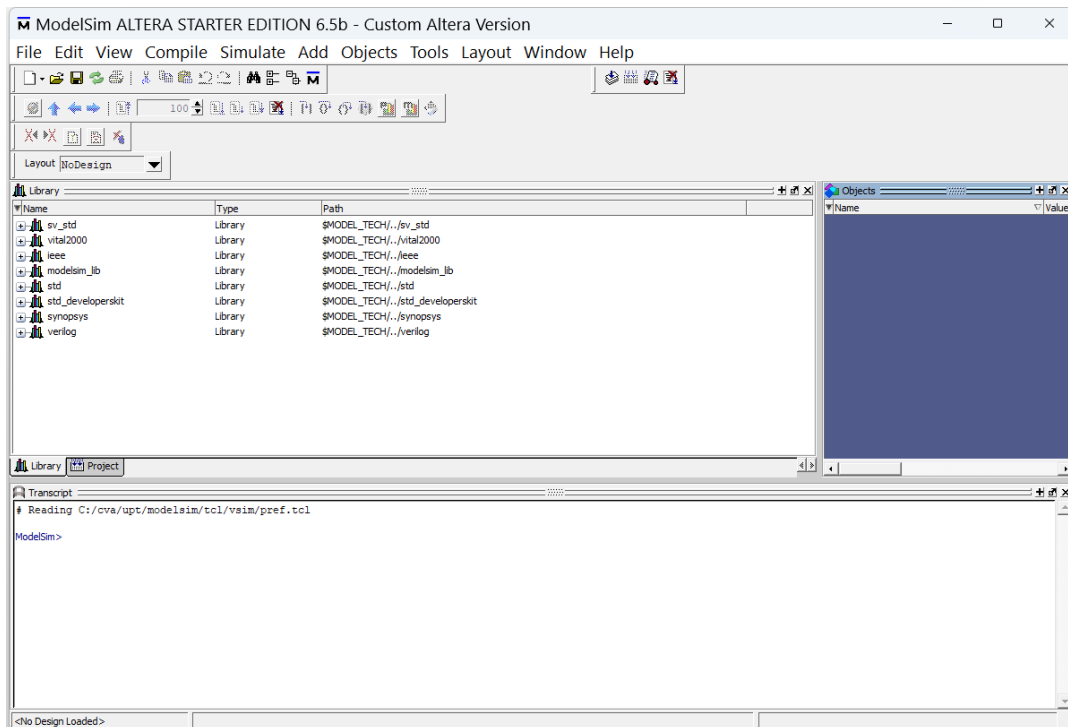
In acest moment trecem la pasul urmator care reprezinta simularea design-ului.

4.1.3 Simularea design-ului “majority voter”

Vom folosi simulatorul ModelSim/QuestaSim software. Dintr-un terminal Linux, lansam ModelSim, folosind comanda:

```
vsim &
```

Simulatorul se va lansa cu o fereastră ca aceasta:



Daca programul deschide un fisier/proiect deja facut, il vom inchide executand: **File->Close** (la aparitia mesajul "This operation will close the current project. Continue?" vom apasa pe "Yes"). Executam **File->Close** pana cand optiunea **Close** devine "griata" si nu mai poate fi executata.

In partea de jos a ferestrei simulatorului, exista o sectiune/ferestra numita *Transcript*, este locul unde vom executa comenzi catre simulator

Daca fereastra "Transcript" nu apare, ea va trebui activata din meniul **View->Transcript**. La fel se pot adauga si alte ferestre, daca lipsesc, cum ar fi: "Library", "Objects", "Project", "Structure", "Wave".

Vom tipari urmatoarele comenzi in fereastra *Transcript*:

cd /<cale>/wa123/a2

- Comanda schimba directorul catre locul unde avem sursele Verilog

vdel -all

- Sterge (daca exista) librariile vechi

vlib work

- Creeaza o librarie de simulare noua, numita *work*

vlog majority_voter.v majority_voter_tb.v

- Compileaza cele doua surse Verilog (design + testbench)

vsim -voptargs=+acc majority_voter_tb

- Seteaza modulul ce trebuie simulat. Observam ca "*majority_voter_tb*" este numele dat **modulului** testului, in *majority_voter_tb.v* (`module majority_voter_tb;`)

add wave *

- Adauga toate semnalele de intrare/iesire pentru a fi vizualizate in formele de unda/diagrama de timp

run 1000ns

- Ruleaza simularea pentru 1000ns.
- Pentru o rulare completa a testului putem folosim **“run -all”**, inasa daca testbench-ul nu are o conditie de oprire, el va rula la infinit si va trebui sa il oprim apasand butonul **“Simulate->Break”**

Fereastra simulatorul va arata in felul urmator:

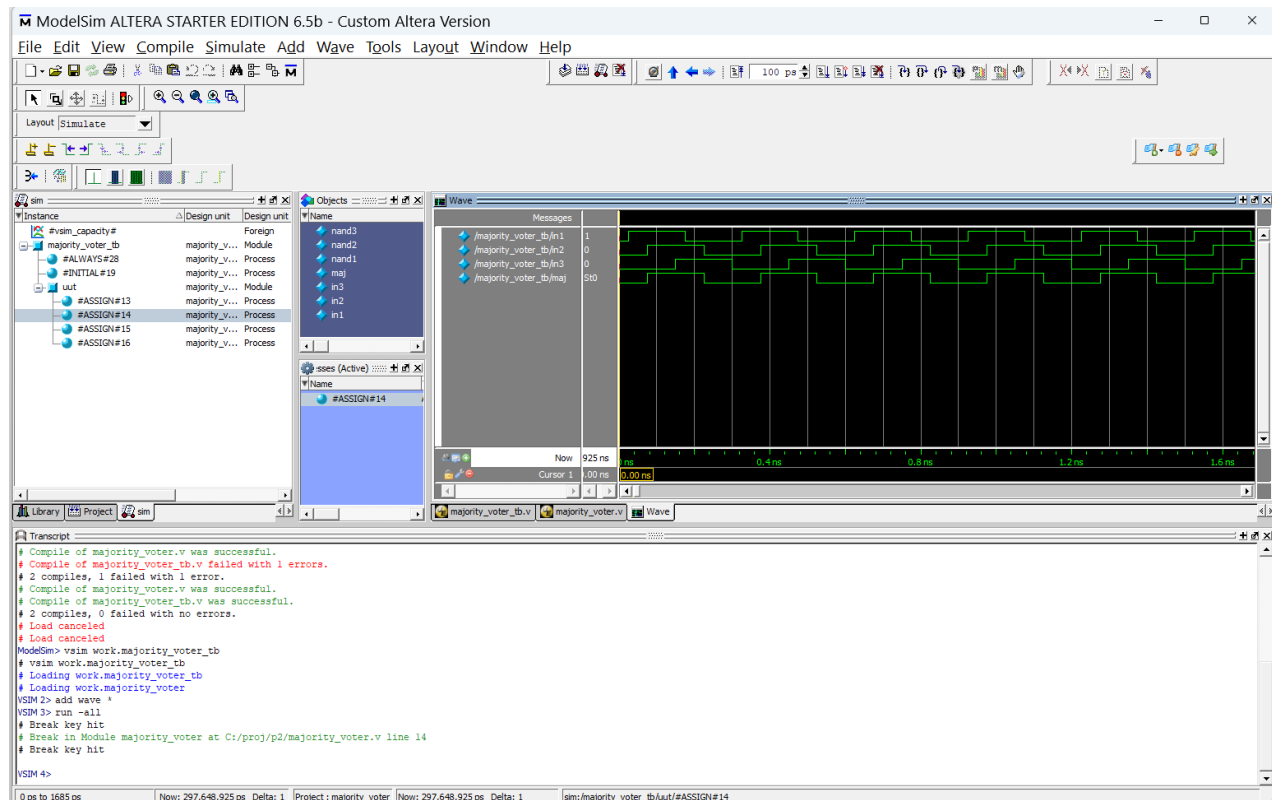
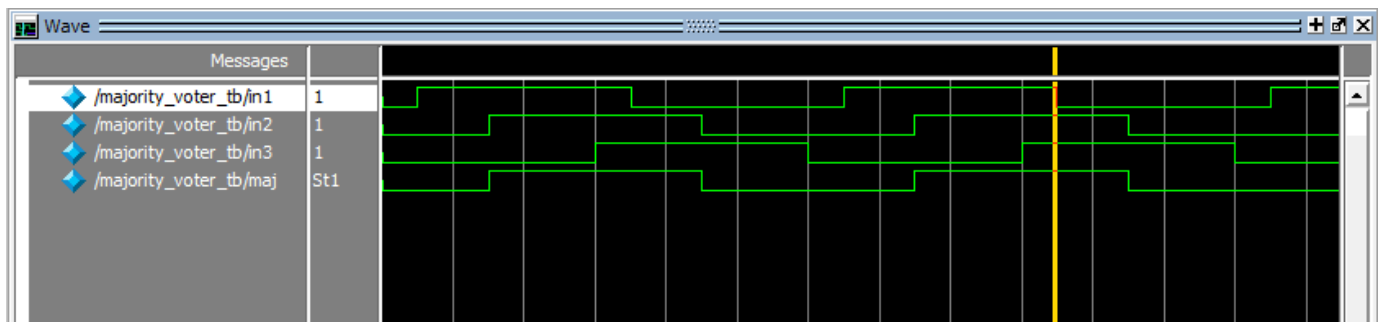


Diagrama de timp aferenta circuitului nostru va arata ca in imagine:



Vom observa cum variaza iesirea *maj* in functie de intrarile *ini1*, *in2*, *in3*.

Daca dorim modificare surselor Verilog si reluarea simularii, vom executa, din nou, urmatoorii pasi:

<code>vlog majority_voter.v majority_voter_tb.v</code>	# re-compilarea surselor
<code>vsim -voptargs=+acc majority_voter_tb</code>	# optional , doar daca modificam numele tb-ului
<code>add wave *</code>	# optional , doar daca am adaugat semnale noi
<code>restart -f</code>	# pentru restartarea simularii de la momentul 0
<code>run 1000ns</code>	

Odata finalizata simularea, vom continua cu implemetarea (capitolul 5).

4.2 Metoda B: Folosirea ModelSim in modul grafic

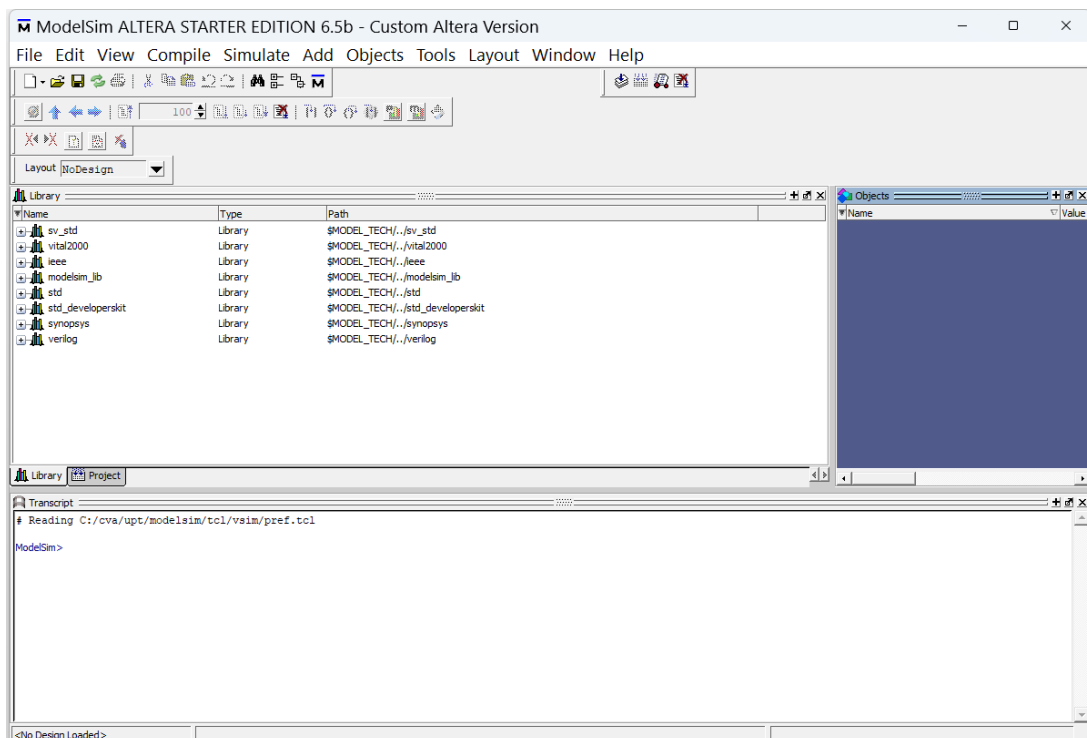
Vom creea structura de directoare asa cum este descrisa in capitolul 4.1.

4.2.1 Descrierea design-ului “majority voter”

Pornim un terminal Linux nou si din el lansam ModelSim, folosind comanda:

`vsim &`

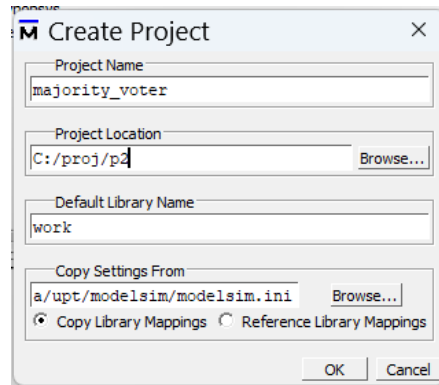
Simulatorul se va lansa cu o fereastra ca aceasta:



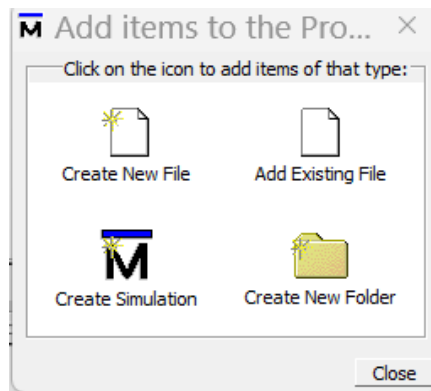
Daca programul deschide un fisier/proiect deja facut, il vom inchide executand: **File->Close** (la aparitia mesajul "This operation will close the current project. Continue?" vom apasa pe "Yes"). Executam **File->Close** pana cand optiunea **Close** devine griata si nu mai poate fi executata.

Creeare proiect nou si fisier sursa de tip Verilog:

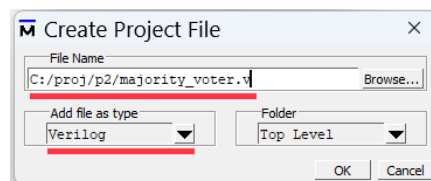
Apasam pe: **File -> New -> Project**. Se va lansa o fereastră nouă, unde completăm numele proiectului si alegem directorul unde va fi salvat:



Apasam "OK", ceea ce se va lansa o nouă fereastră unde putem adăuga fișiere noi:



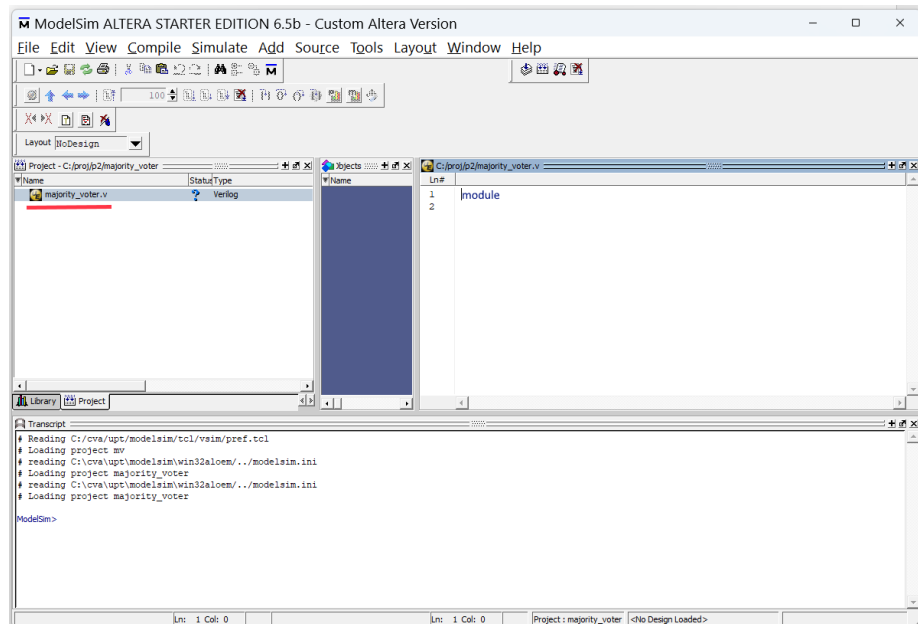
Apasam pe **"Create New File"** (sau **"Add Existing File"** dacă deja avem un fișier făcut în prealabil):



Completăm numele fișierului (exemplu *"majority_voter.v"*) și îl plasăm în același director unde am creat proiectul (în exemplul din imagine este *"C:/proj/p2"*). Ne asigurăm că fișierul este de tip *Verilog*, ca în imagine. Apasăm "OK" și închidem fereastra *"Add items to .."* apăsând "Close" (noul fișier va apărea în fereastra intitulată *"Project"*).

Editare fisier sursa “majority_voter.v”:

Pentru editare facem dublu-click pe nume fisier in fereastra “Project” - “Name” si se va deschide ca fisier nou in editorul de text.



Codul aferent este cel descris in capitolul **4.1.1** (module majority_voter ..). Dupa editare il salvam (**File->Save**).

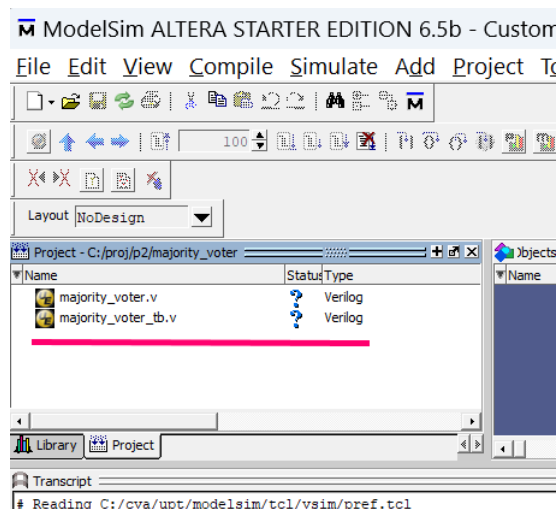
4.2.2 Descrierea testului pentru “majority voter”

Vom adauga o sursa noua la proiect, apasand: **File->New->Source->Verilog**. Editam fisierul nou creat, dupa cum este descris in capitolul **4.1.2** (*module majority voter tb ..*)

Dupa editare, il salvam apasand pe **File->Save As** si alegand numele *majority_voter_tb.v*.

Apoi adaugam acest fisier la proiectul nostru apasand pe **Project->Add to Project -> Existing File**. Se va deschide o mica fereastră unde vom apasa **Browse**, selectam fisierul de testbench si apasam **OK**.

In fereastra *Project* vor aparea cele doua fisere, ca in imagine:



Daca unul sau mai multe fisiere necesare proiectului nu apar in aceasta lista, va trebui sa le includem manual apasand pe **Project->Add to Project -> Existing File**.

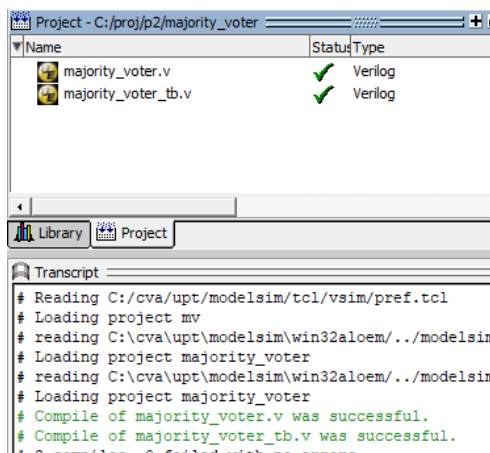
Semnul intrebării (?) din dreptul fisierelor ne arata ca ele nu au fost inca “compile” pentru simulare. Acesta se va face in pasul urmator.

In acest moment trecem la pasul urmator care reprezinta compilarea si simularea design-ului.

4.2.3 Simularea design-ului “majority voter”

Compilarea surselor:

Apasam **Compile->Compile All**. Daca nu au fost intampinate erori, semnul intrebării din dreptul fisierelor va fi inlocuit de un “V”. Atunci suntem gata de simulare.

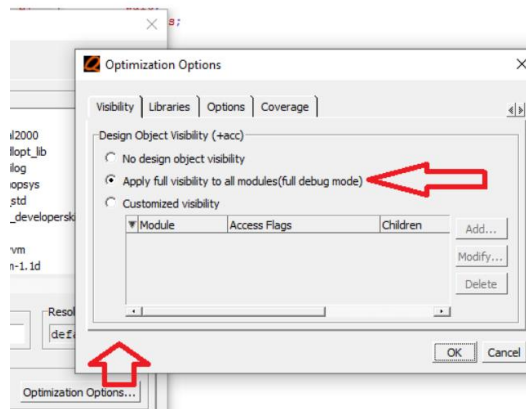
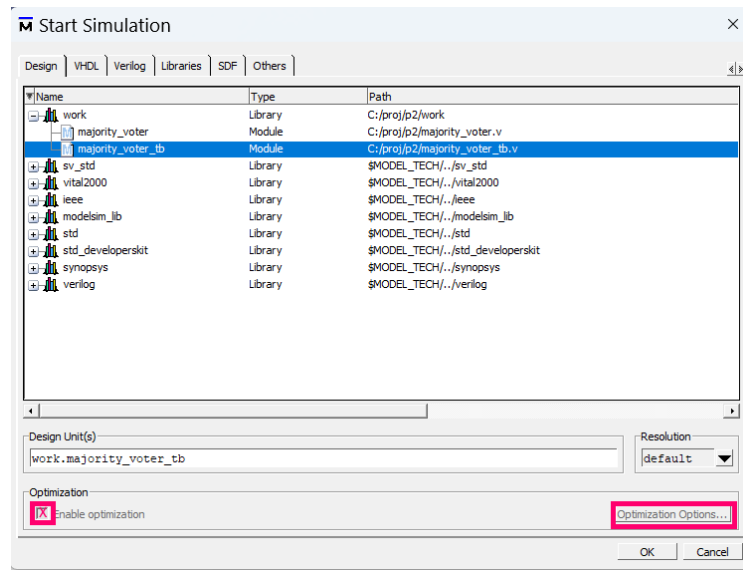


Daca au fost detectate erori, in dreptul fisierului va aprea un “X” si un mesaj in fereastra *Transcript* ne va raporta (cu rosu) acest lucru, precum si fisierul ce contine erori. Va trebui sa corectam erorile, sa re-savam fisierul in cauza, si sa executam din nou compilarea pana ce nu mai sunt erori si compilarea are succes.

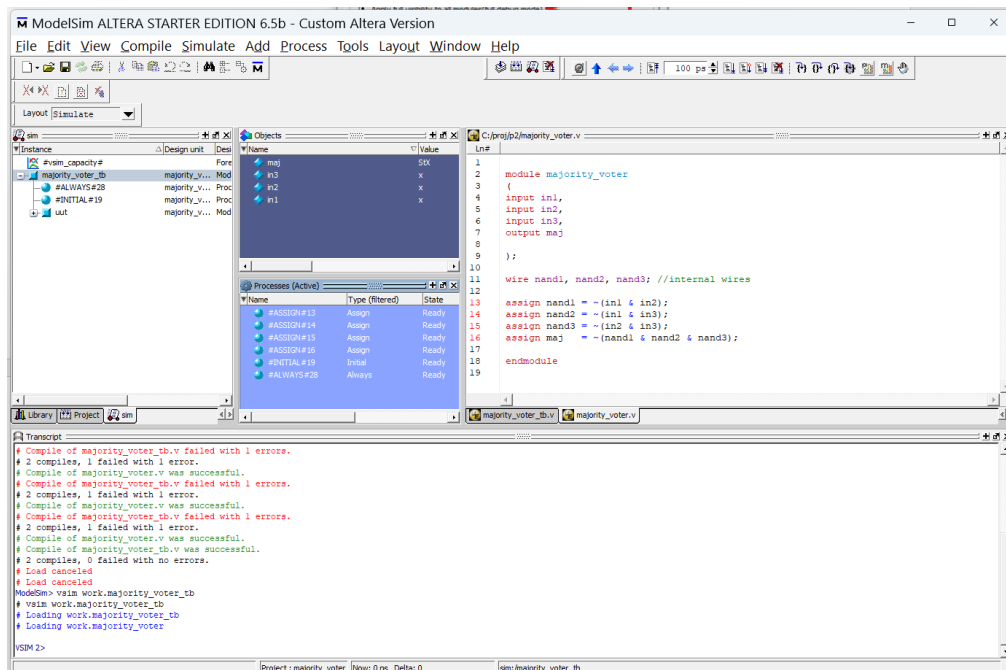
Simularea:

Apasam **Simulate->Start Simulation**. Se va lansa o fereastra ca in imaginea de mai jos, de unde va trebui sa selectam modulul ce trebuie simulat (top level). Selectam **work -> majority_voter_tb**.

Apoi bifam optiunea *Enable optimization* si apasam pe *Optimization Options*. In fereastra nou lansata selectam *Apply full visibility to all modules(full debug mode)* in tab-ul *Visibility*, apasam “OK” si “OK”.



In acest moment fereastra programului va arata asa:

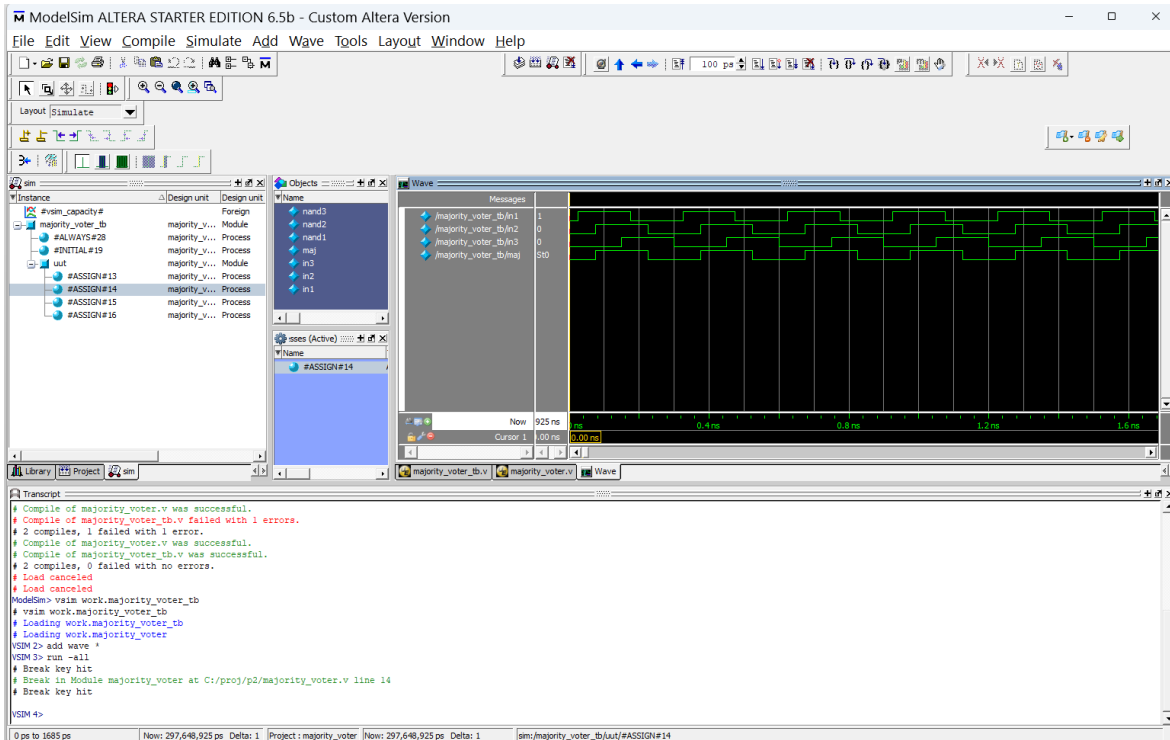


Apasam **View -> Wave** pentru a afisa fereastra diagramei de timp (Wave), si vom vom “trage” in ea semnalele din fereastra *Objects* pentru a fi urmarite. Alternativ, putem adauga semnalele tiparind urmatoarea comanda in fereastra *Transcript*: **add wave ***

Apoi pornim simularea apasand pe *Simulate->Run->Run 100* sau *Simulate->Run->Run All*. Alternativ, putem lansa simularea executand in fereastra *Transcript*: **run -all**

Daca selectam “run -all” si nu am pus nici o conditie de oprire in testbench, simularea ruleaza la infinit. Va trebui oprita apasand Simulate -> Break

Fereastra simulatorul va arata in felul urmator:



*Daca fereastra “Wave” nu apare, ea va trebui activata din meniul **View -> Wave**. La fel se pot adaga si alte ferestre, daca lipsesc, cum ar fi “Library”, “Objects”, “Project”, “Structure”, “Transcript”.*

*Pentru resetarea ferestrelor ca dimensiuni, pozitie, se va selecta **Layout->Reset**.*

5. Sinteza si implementarea design-ului

Vom folosi Quartus pentru sinteza si implementare si vom lucra in modul “linie de comanda” (script).

Design-ul care trebuie implementat pe placa FPGA este “majority_voter.v”. Trebuie observant ca “majority_voter_tb.v” a fost folosit doar pentru simulare, deci va fi abandonat din acest moment.

Toate intrarile si iesirile design-ului trebuiesc asignate catre pinii corespunzatori:

- Semnalele de intrare: *in1*, *in2*, *in3* le vom conecta la intrerupatoarele SW0, SW1, SW2, care sunt conectate la FPGA la pinii *PIN_C10*, *PIN_C11*, *PIN_D12*.
- Semnalele de iesire: *maj* il vom conecta la LED-ul LEDR1, conectat la FPGA la pinul *PIN_A9*.

Vom crea un fisier text nou, pe care il vom salva in directorul unde am pus si sursele design-ului (“majority_voter.v”). Il vom numi “prj.tcl” si va avea urmatorul continut:

```
project_new example1 -overwrite
set_global_assignment -name FAMILY MAX10
set_global_assignment -name DEVICE 10M50DAF484C7G
set_global_assignment -name VERILOG_FILE majority_voter.v
set_global_assignment -name TOP_LEVEL_ENTITY majority_voter
set_location_assignment -to in1 PIN_C10
set_location_assignment -to in2 PIN_C11
set_location_assignment -to in3 PIN_D12
set_location_assignment -to maj PIN_A9
load_package flow
execute_flow -compile
project_close
```

Acest fisier reprezinta script-ul folosit de Quartus pentru rularea compilarii design-ului si generarea fisierului de programare/configurare (.sof).

Apoi, din terminalul LINUX vom naviga in directorul unde am pus sursele design-ului si implicit “prj.tcl”:

```
cd nume_director
```

Si vom executam comanda:

```
quartus_sh -t prj.tcl
```

Comanda va rula compilarea in Quartus, in modul script (batch mode). Daca au fost intampinate erori, programul le va raporta. Daca nu, iar compilarea a avut success, fisierul de programare va fi prezent in directorul current. Pentru a verifica existenta fisierul de programare (.sof) executam comanda:

```
ls -al
```

sau

```
ls -al *.sof
```

6. Configurare/programare FPGA

Conectam cablul USB la placa DE10-Lite si la PC.

Tastam urmatoarele comenzi:

```
quartus_pgm -l
```

```
quartus_pgm --auto
```

```
quartus_pgm -m JTAG -o "p;example1.sof@1"
```

Comenzile "quartus_pgm -l" si "quartus_pgm --auto" vor afisa lista conexiunilor/dispozitivelor FPGA detectate.

Comanda quartus_pgm -m JTAG -o "p;example1.sof@1" reprezinta, efectiv, comanda de programare/configurare a FPGA-ului.

7. Exercitii

Realizati pașii indicați. Completați liniile de cod lipsă. Realizați design-ul și construiți tabelul de adevăr pentru o poartă majority voting cu 5 intrări.

Verificați funcționarea corectă a design-ului pe placă!